



Module Code	:	CT065-3-3-ALG
Intake Code	:	APD3F2601CS(CYB)
Assignment	:	Group Assignment 1: Algorithmic Performance and Evaluation
Title	:	Evaluation of Disjoint Set Union (Union-Find) Using Find with Path Compression and Dynamic Connectivity
Lecturer Name	:	Maznah Binti Mohd Zin
Hand in Date	:	10 th May 2026

Student ID	Student Name
TP070844	Hemanthrajah A/L N Krishnarajah
TP071154	Anusha Devy A/P Brahmataran
TP070071	Darmik Ishwar A/L Ravi
TP070581	Srisudharson A/L Perumal

Table of Contents

1.0 Introduction.....	5
Part A: Description and Representation	6
2.1 Data Structure Description: Disjoint Set Union	6
2.1.1 Definition and Purpose.....	6
2.1.2 Logical Organization and Storage.....	6
2.1.3 Advantages, Disadvantages, and Applications	8
2.2 Algorithm Representation	9
2.2.1 Algorithm 1: Find with Path Compression.....	9
2.2.2 Algorithm 2: Dynamic Connectivity using Union-Find	10
Part B: Algorithmic Correctness Demonstration	12
3.1 Selected Best-Case and Worst-Case Input Data Sets	12
3.1.1 Best-Case Input Data Set.....	12
3.1.2 Worst-Case Input Data Set.....	13
3.2 Correctness Demonstration for Algorithm 1.....	15
3.2.1 Worst-Case Trace for FIND_PC(1).....	15
3.3 Correctness Demonstration for Algorithm 2.....	17
3.3.1 Worst-Case Trace for Dynamic Connectivity	17
Part C: Asymptotic Complexity Analysis.....	19
4.1 Complexity Analysis for Algorithm 1: Find with Path Compression	19
4.1.1 Worst-Case Complexity.....	19
4.1.2 Best-Case Complexity	19
4.1.3 Best-Case Trace.....	20
4.2 Complexity Analysis for Algorithm 2: Dynamic Connectivity.....	20
4.2.1 Worst-Case Complexity.....	20
4.2.2 Best-Case Complexity	21
4.2.3 Best-Case Trace	21
Part D: Comparative Evaluation and Conclusion.....	22
5.1 Comparative Evaluation	22
5.2 Conclusion.....	24
Part E: Problem Solving Skill.....	25
7.0 References	26
8.0 Appendix A: Requirement Mapping Checklist.....	27

List of Figures

<i>Figure 1: Example DSU forest with three components</i>	<i>7</i>
<i>Figure 2: Pseudocode Implementation of Find with Path Compression Algorithm</i>	<i>9</i>
<i>Figure 3: Pseudocode Implementation of Dynamic Connectivity using Union-Find</i>	<i>10</i>
<i>Figure 4: Continuation of Pseudocode Implementation of Dynamic Connectivity using Union-Find</i>	<i>11</i>
<i>Figure 5: Parent chain before path compression.....</i>	<i>13</i>
<i>Figure 6: Parent structure after path compression.....</i>	<i>16</i>
<i>Figure 7: Dynamic connectivity DSU state before the final query</i>	<i>18</i>

List of Tables

<i>Table 1: Logical representation of DSU parent and rank arrays</i>	<i>7</i>
<i>Table 2: Selected best-case input data set.....</i>	<i>12</i>
<i>Table 3: Selected worst-case input data set for Algorithm 1</i>	<i>13</i>
<i>Table 4: Selected worst-case dynamic connectivity sequence</i>	<i>14</i>
<i>Table 5: Worst-case trace for Find with Path Compression.....</i>	<i>15</i>
<i>Table 6: Dynamic connectivity operation sequence and trace</i>	<i>17</i>
<i>Table 7: Best-case trace for Find with Path Compression</i>	<i>20</i>
<i>Table 8: Best-case trace for Dynamic Connectivity.....</i>	<i>21</i>
<i>Table 9: Comparative complexity summary.....</i>	<i>21</i>
<i>Table 10: Requirement mapping checklist</i>	<i>27</i>

1.0 Introduction

Designing efficient algorithms is significant since a computer program should not only give the correct result, but it should run within the acceptable time and memory constraints. Many computing problems involve grouping items, merging groups when a relationship is discovered, and repeatedly checking whether two items belong to the same connected component. This type of requirement appears in computer networks, social networks, road systems, clustering, image segmentation, and graph-based optimization.

For this report, the selected data structure is Disjoint Set Union (DSU), also known as Union-Find. The selected algorithms are Find with Path Compression and Dynamic Connectivity using Union-Find. This pairing is suitable because both algorithms are directly related to the same parent-array structure, but they operate at different levels. Find with Path Compression is a core DSU operation that locates the representative of a set while improving the internal tree shape. Dynamic Connectivity is a higher-level algorithmic procedure that uses DSU operations to process a changing sequence of connection and query operations.

The purpose of this report is to evaluate the chosen data structure and algorithms through description, representation, correctness demonstration, asymptotic complexity analysis, and comparative evaluation. The report also explains the selected best-case and worst-case input data sets used for tracing. This structure follows the assignment requirement to select one data structure, two related algorithms, and two specific input data sets for best-case and worst-case analysis.

Part A: Description and Representation

2.1 Data Structure Description: Disjoint Set Union

2.1.1 Definition and Purpose

Disjoint Set Union is a data structure which is used to keep a collection of disjoint sets. There is only one set to which each element belongs and each set has one element, called the root, that represents it. The operations the structure supports are: Find the representative of the set containing an element and Merge two sets into one. These operations makes DSU particularly useful for issues of connectivity.

Values are not stored in sorted order in a DSU. Rather, it holds membership relationships. Elements that end up at the same root via parents are in the same set. If they don't lead to the same root, then they are in different sets. This stems from the root approach that enables answering connectivity questions without any need to scan all the elements again.

2.1.2 Logical Organization and Storage

A forest of rooted trees is the logical organization of DSU. Every tree is a disjoint set, with root of the tree as representative of the set. All nodes contain reference to its parent node. When a node is the root, it has a pointer to itself. The following parent-pointer approach is a traditional approach to implementing union-find structures and can be simply represented with arrays (cp-algorithms, 2026; Linux Kernel Documentation, 2024).

The main storage arrays are parent [] and rank []. The parent array stores the parent of each element. The rank array stores an estimate of the tree height and is used by union-by-rank to keep the tree shallow. Although rank is not the main selected algorithm, it is used as a supporting implementation detail in the dynamic connectivity procedure to avoid poor merging decisions.

Element	1	2	3	4	5	6
parent	2	4	5	4	5	6
rank	0	1	0	2	1	0

Table 1: Logical representation of DSU parent and rank arrays

In Table 1, elements 1, 2, and 4 belong to one set because 1 points to 2, 2 points to 4, and 4 points to itself. Elements 3 and 5 belong to another set because 3 points to 5 and 5 points to itself. Element 6 is alone because it points to itself. The following figure illustrates the idea visually.

Example DSU Forest with Three Components

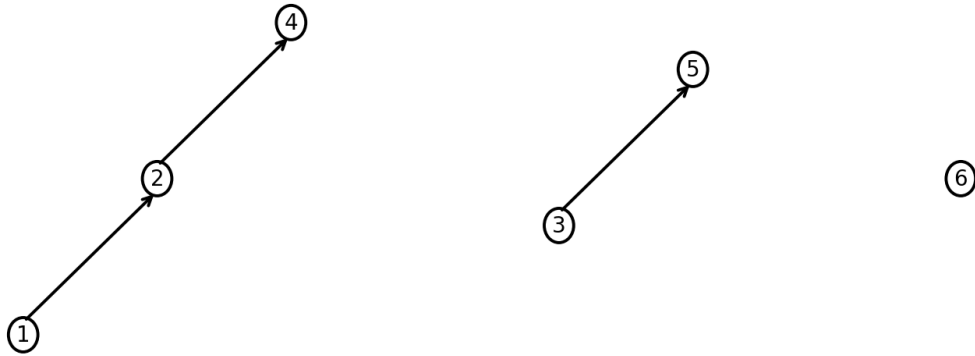


Figure 1: Example DSU forest with three components

2.1.3 Advantages, Disadvantages, and Applications

The first advantage of DSU is efficiency. It avoids repeatedly rebuilding connected components from scratch. This is valuable when many updates and queries are processed. The second advantage is its compact storage, because only parent and rank arrays are needed. The third advantage is that it naturally supports graph connectivity applications, including cycle detection and Kruskal-style minimum spanning tree construction.

However, DSU also has limitations. It is not suitable for ordered searching, range queries, or arbitrary deletion-heavy workloads. It is specialized for grouping and connectivity. Another limitation is that its strongest performance behavior is best explained using amortized analysis, which is more advanced than simple single-operation analysis. Even so, it remains a suitable data structure for this assignment because its operations can be traced clearly and its complexity can be justified using Big-O notation.

Common applications include network connectivity, social-group detection, image segmentation, clustering, and graph algorithms. Recent union-find learning materials describe dynamic connectivity as maintaining a graph while adding connections and answering whether two vertices are connected (DTU Compute, 2023; Labuladong, 2026).

2.2 Algorithm Representation

2.2.1 Algorithm 1: Find with Path Compression

The purpose of Find with Path Compression is to return the representative of the set containing a given element x . During this process, the algorithm also compresses the path from x to the root by making every visited node point directly to the root. This improves the speed of future find operations because the tree becomes flatter after the call.

The pseudocode below shows the recursive version of the algorithm. The base case occurs when the element is already its own parent. Otherwise, the algorithm recursively finds the root of the parent and then assigns that root directly to $\text{parent}[x]$.

```
FIND_PC(x):  
    if parent[x] == x:  
        return x  
  
    parent[x] = FIND_PC(parent[x])  
    return parent[x]
```

Figure 2: Pseudocode Implementation of Find with Path Compression Algorithm

The important correctness idea is that the returned root is the same root that would be reached by ordinary parent traversal. Path compression does not change which set an element belongs to; it only improves the internal representation of that set.

2.2.2 Algorithm 2: Dynamic Connectivity using Union-Find

Dynamic Connectivity processes a sequence of operations on a changing system. In this report, each operation is either `connect(u, v)`, which adds a connection between two elements, or `query(u, v)`, which asks whether the two elements are currently connected. The algorithm uses DSU to store connected components incrementally.

Although Dynamic Connectivity is a higher-level problem, it can be represented as an algorithm because it has a clear input, process, and output. Its input is a number of elements and a sequence of operations. Its output is the result of each query operation. Internally, it calls Find with Path Compression and a union-by-rank merge to keep the DSU forest shallow.

```
MAKE_SET(n):
    for i = 1 to n:
        parent[i] = i
        rank[i] = 0

FIND_PC(x):
    if parent[x] == x:
        return x
    parent[x] = FIND_PC(parent[x])
    return parent[x]

UNION_BY_RANK(x, y):
    rootX = FIND_PC(x)
    rootY = FIND_PC(y)

    if rootX == rootY:
        return
```

Figure 3: Pseudocode Implementation of Dynamic Connectivity using Union-Find

```

    if rank[rootX] < rank[rootY]:
        parent[rootX] = rootY
    else if rank[rootX] > rank[rootY]:
        parent[rootY] = rootX
    else:
        parent[rootY] = rootX
        rank[rootX] = rank[rootX] + 1

DYNAMIC_CONNECTIVITY(n, operations):
    MAKE_SET(n)

    for each operation op in operations:
        if op.type == "connect":
            UNION_BY_RANK(op.u, op.v)
        else if op.type == "query":
            if FIND_PC(op.u) == FIND_PC(op.v):
                output "Connected"
            else:
                output "Not Connected"

```

Figure 4: Continuation of Pseudocode Implementation of Dynamic Connectivity using Union-Find

For a connect operation, the algorithm merges the components containing u and v . For a query operation, it compares the representatives of u and v . If the representatives are the same, the elements are connected; otherwise, they are not connected. This approach is much more efficient than recomputing all connected components after every update.

Part B: Algorithmic Correctness Demonstration

This section demonstrates correctness using the selected best-case and worst-case input data sets. The worst-case traces show every major logical step of each algorithm. The best-case traces are included later in Part C because the assignment asks for them together with the best-case complexity justification.

3.1 Selected Best-Case and Worst-Case Input Data Sets

3.1.1 Best-Case Input Data Set

Item	Selected Data
Elements	{1, 2, 3, 4, 5, 6, 7, 8}
Initial parent array	parent[i] = i for every element
Initial rank array	rank[i] = 0 for every element
Algorithm 1 operation	FIND_PC(3)
Algorithm 2 operation	query(4, 4)
Reason for best case	The searched element is already a root, and the dynamic query compares an element with itself.

Table 2: Selected best-case input data set

The best-case data set is designed so that both algorithms terminate through their shortest possible execution paths. In the first algorithm, element 3 is already its own representative. In the second algorithm, query(4, 4) is immediately true because an element is always connected to itself.

3.1.2 Worst-Case Input Data Set

Element	1	2	3	4	5	6	7
parent	2	3	4	5	6	7	7

Table 3: Selected worst-case input data set for Algorithm 1

For Algorithm 1, the worst-case parent structure is the chain 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7. The operation FIND_PC(1) begins at the deepest node and must follow all parent links before reaching the root.

Before FIND_PC(1): Linear Parent Chain

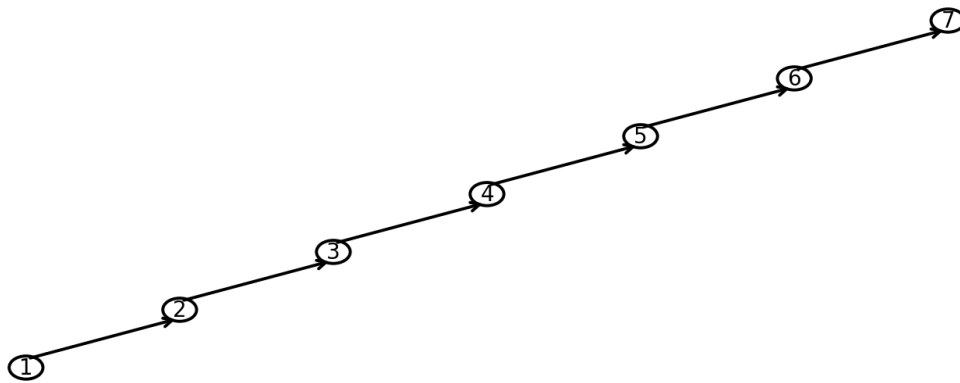


Figure 5: Parent chain before path compression

For Algorithm 2, the worst-case dynamic connectivity sequence uses eight elements and builds a rank-balanced DSU tree before the final query. This creates the maximum height possible under union-by-rank for eight elements, which is $\log_2(8) = 3$ before the final compression step.

Step	Operation	Purpose
1	connect(1, 2)	Build first pair
2	connect(3, 4)	Build second pair
3	connect(5, 6)	Build third pair
4	connect(7, 8)	Build fourth pair
5	connect(1, 3)	Merge two rank-1 components
6	connect(5, 7)	Merge two rank-1 components
7	connect(1, 5)	Merge two rank-2 components
8	query(8, 4)	Query two deep leaves before compression

Table 4: Selected worst-case dynamic connectivity sequence

3.2 Correctness Demonstration for Algorithm 1

3.2.1 Worst-Case Trace for FIND_PC(1)

Step	Active Call	Observation	Action / Update	Return
1	FIND_PC(1)	parent[1] = 2, so 1 is not root	Call FIND_PC(2)	Pending
2	FIND_PC(2)	parent[2] = 3, so 2 is not root	Call FIND_PC(3)	Pending
3	FIND_PC(3)	parent[3] = 4, so 3 is not root	Call FIND_PC(4)	Pending
4	FIND_PC(4)	parent[4] = 5, so 4 is not root	Call FIND_PC(5)	Pending
5	FIND_PC(5)	parent[5] = 6, so 5 is not root	Call FIND_PC(6)	Pending
6	FIND_PC(6)	parent[6] = 7, so 6 is not root	Call FIND_PC(7)	Pending
7	FIND_PC(7)	parent[7] = 7, so 7 is root	Root found	7
8	Return to 6	Root is known to be 7	Set parent[6] = 7	7
9	Return to 5	Root is known to be 7	Set parent[5] = 7	7
10	Return to 4	Root is known to be 7	Set parent[4] = 7	7
11	Return to 3	Root is known to be 7	Set parent[3] = 7	7
12	Return to 2	Root is known to be 7	Set parent[2] = 7	7
13	Return to 1	Root is known to be 7	Set parent[1] = 7	7

Table 5: Worst-case trace for Find with Path Compression

The final returned value is 7, so the algorithm correctly identifies 7 as the representative of the set containing 1. The final parent array becomes $\text{parent}[1] = \text{parent}[2] = \text{parent}[3] = \text{parent}[4] = \text{parent}[5] = \text{parent}[6] = \text{parent}[7] = 7$. This means all visited nodes now point directly to the representative.

This data set leads to the worst case because the queried element is at the deepest possible position in the parent structure. Before compression, the algorithm must visit every node on the path. The correctness of the algorithm follows from the invariant that every recursive call moves one parent link closer to the same root. When recursion returns, each visited node is linked directly to that root, but no element is moved into a different set.

After FIND_PC(1): Path Compressed to Root 7

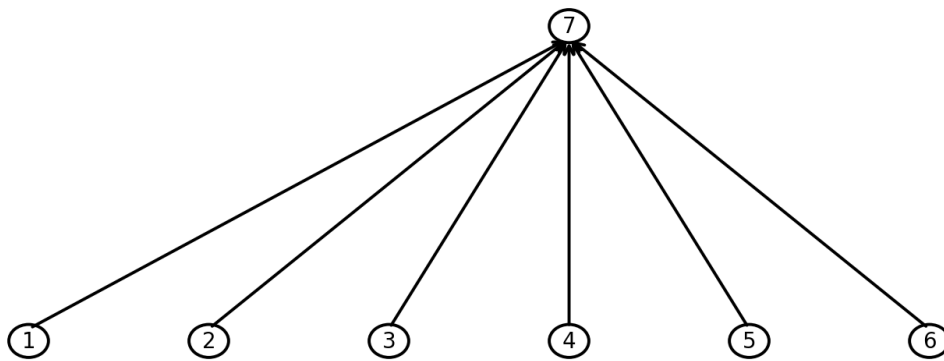


Figure 6: Parent structure after path compression

3.3 Correctness Demonstration for Algorithm 2

3.3.1 Worst-Case Trace for Dynamic Connectivity

The dynamic connectivity trace starts with eight isolated elements. Each connect operation merges components, while the final query checks whether elements 8 and 4 are connected. The sequence is designed so that the final query reaches deep leaves before path compression occurs.

Step	Operation	Root Finding / Decision	Parent and Rank Effect	Output
0	Initial state	Each node is its own root	$\text{parent}[i] = i; \text{rank}[i] = 0$	-
1	connect(1,2)	Roots 1 and 2 have equal rank	$\text{parent}[2] = 1; \text{rank}[1] = 1$	-
2	connect(3,4)	Roots 3 and 4 have equal rank	$\text{parent}[4] = 3; \text{rank}[3] = 1$	-
3	connect(5,6)	Roots 5 and 6 have equal rank	$\text{parent}[6] = 5; \text{rank}[5] = 1$	-
4	connect(7,8)	Roots 7 and 8 have equal rank	$\text{parent}[8] = 7; \text{rank}[7] = 1$	-
5	connect(1,3)	Roots 1 and 3 have equal rank 1	$\text{parent}[3] = 1; \text{rank}[1] = 2$	-
6	connect(5,7)	Roots 5 and 7 have equal rank 1	$\text{parent}[7] = 5; \text{rank}[5] = 2$	-
7	connect(1,5)	Roots 1 and 5 have equal rank 2	$\text{parent}[5] = 1; \text{rank}[1] = 3$	-
8	query(8,4)	FIND_PC(8): 8 -> 7 -> 5 -> 1; FIND_PC(4): 4 -> 3 -> 1	Compress 8, 7, 5, 4, and 3 toward root 1	Connected

Table 6: Dynamic connectivity operation sequence and trace

Before the final query, the DSU has one component whose root is 1. Some nodes are still not directly attached to 1, which means the query demonstrates non-trivial find paths. In particular, 8 reaches the root through 7 and 5, while 4 reaches the root through 3. After the final query, path compression shortens both paths by attaching the visited nodes closer to root 1.

The final query correctly returns Connected because both $\text{FIND_PC}(8)$ and $\text{FIND_PC}(4)$ return root 1. This is correct according to the meaning of dynamic connectivity: after the seven connect operations, there is a chain of added relationships connecting every node from 1 to 8 into one component.

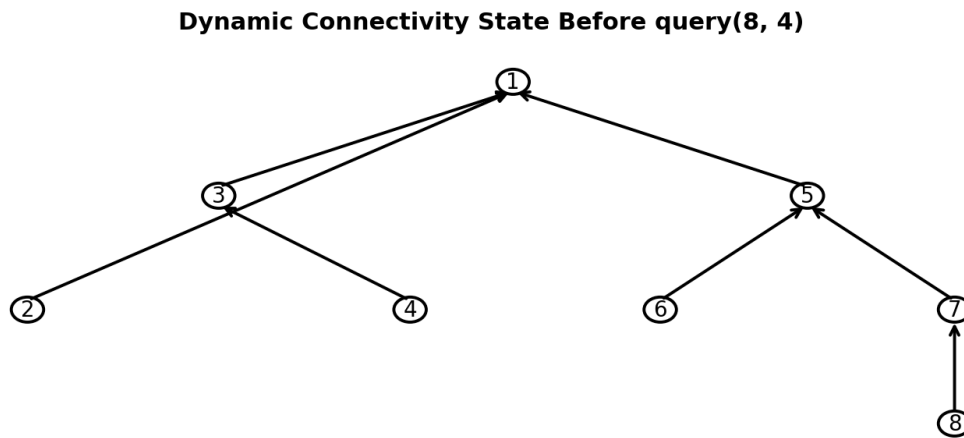


Figure 7: Dynamic connectivity DSU state before the final query

A stronger correctness argument can be stated using an invariant. After processing the first k operations, two elements are in the same DSU set if and only if they are connected by the connect operations processed so far. The base case is true before any operation because every element is isolated. A connect operation preserves the invariant by merging exactly the two components joined by the new connection. A query operation is correct because it compares the representatives of the two elements. Therefore, the dynamic connectivity algorithm remains correct throughout the operation sequence.

Part C: Asymptotic Complexity Analysis

4.1 Complexity Analysis for Algorithm 1: Find with Path Compression

4.1.1 Worst-Case Complexity

For Algorithm 1, the worst-case structure is a linear parent chain of n elements. If `FIND_PC` is called on the deepest node, it must follow all parent pointers to reach the root. The work can be expressed as a summation of constant-time checks and recursive movements along the chain:

$T(n) = c + c + c + \dots + c$ for n visited nodes $= cn$. After removing the constant multiplier, the dominant term is n . Therefore, the worst-case time complexity of a single isolated Find with Path Compression on an uncompressed chain is $O(n)$.

This worst case is reached before compression takes effect. After the operation finishes, the same path becomes flat, so later find operations on those nodes become faster. This is why path compression is powerful in repeated use, even though one isolated worst-case call can still be linear.

4.1.2 Best-Case Complexity

The best case occurs when the searched element is already the representative of its own set. The algorithm only checks whether `parent[x] == x` and immediately returns `x`. Therefore, $T(n) = c$, where c is a constant amount of work. The best-case complexity is $O(1)$.

4.1.3 Best-Case Trace

Step	Operation	Observation	Result
1	FIND_PC(3)	parent[3] = 3	3 is already the root
2	Return	No recursion needed	Return 3

Table 7: Best-case trace for Find with Path Compression

This trace directly supports $O(1)$ because only one condition is checked and no parent traversal is performed.

4.2 Complexity Analysis for Algorithm 2: Dynamic Connectivity

4.2.1 Worst-Case Complexity

Dynamic Connectivity processes a sequence of operations. Let n be the number of elements and m be the number of operations. A query operation performs two find operations. A connect operation also performs two find operations and then at most one constant-time parent update. Since the algorithm uses union by rank as a supporting merge rule, the height of a DSU tree before compression is at most logarithmic in the number of elements. Therefore, a single operation is bounded by $O(\log n)$ in the conservative worst-case analysis.

For one operation, the cost can be written as $T_{\text{op}}(n) \leq 2(c \log^2 n) + d$. The dominant term is $\log n$, so $T_{\text{op}}(n) = O(\log n)$. For m operations, the total cost is $T(m, n) = \sum_{i=1}^m O(\log n) = O(m \log n)$. The dominant term for the full operation sequence is $m \log n$.

With both path compression and union by rank used together over a long sequence, the well-known amortized bound is even stronger: $O(m \alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function and grows extremely slowly (cp-algorithms, 2026; GeeksforGeeks, 2026). For this assignment, the $O(m \log n)$ bound is a conservative worst-case explanation that is easier to connect directly to the trace, while the amortized note shows deeper understanding of optimized DSU behavior.

4.2.2 Best-Case Complexity

The best case occurs when the dynamic operation requires no meaningful traversal. For example, `query(4, 4)` checks whether an element is connected to itself. If 4 is already its own root, both find calls return immediately. Thus, one best-case operation is $O(1)$. For m best-case operations, the total cost is $O(m)$.

4.2.3 Best-Case Trace

Step	Operation	Observation	Result
1	<code>query(4,4)</code>	Call <code>FIND_PC(4)</code>	Returns 4 immediately
2	<code>query(4,4)</code>	Call <code>FIND_PC(4)</code> again	Returns 4 immediately
3	Compare roots	$4 == 4$	Output Connected

Table 8: Best-case trace for Dynamic Connectivity

This trace supports $O(1)$ for a single best-case dynamic operation because only a constant number of root checks and comparisons are performed.

Algorithm	Best Case	Worst Case	Main Reason
Find with Path Compression	$O(1)$	$O(n)$ for a single uncompressed chain	A root is found immediately in the best case; a chain may require visiting every node in the worst case.
Dynamic Connectivity using DSU	$O(1)$ per trivial operation	$O(\log n)$ per operation, $O(m \log n)$ for m operations; amortized $O(m \alpha(n))$ with full DSU optimization	Each update or query performs a small number of DSU root searches. Union by rank limits height, and path compression improves repeated operations.

Table 9: Comparative complexity summary

Part D: Comparative Evaluation and Conclusion

5.1 Comparative Evaluation

Although Algorithm 1 and Algorithm 2 are similar, there are a few differences between the two. Path Compression is an essential operation of the DSU structure. Dynamic Connectivity: A more advanced algorithm (using DSU) to solve a connectivity problem that changes over time. This implies that Algorithm 1 is application specific and Algorithm 2 is more general.

As to the performance, Algorithm 1 is typically less expensive for a single direct task since it requires only one representative to be found. Its worst case is $O(n)$ just for the case when the input structure is a long uncompressed chain, its best case is $O(1)$. After the path has been compressed, subsequent work on the same nodes is executed very much faster. The total cost of Algorithm 2 is higher as it handles an entire sequence of updates and queries. But there is a trade-off: this is an expensive solution, and one that is a necessity for a larger issue: connectivity over time.

For small input sizes, both algorithms are efficient. The difference becomes more important as n and m increase. If the system only needs to know the representative of one element, Algorithm 1 is more suitable. If the system must support many connections and connectivity queries, Algorithm 2 is more suitable because it avoids repeatedly running full graph traversal algorithms after every update.

Data characteristics also affect performance. A long uncompressed chain is difficult for Find with Path Compression during the first call, but it becomes easy after compression. In Dynamic Connectivity, union by rank prevents the structure from becoming too deep, while path compression improves repeated queries. Therefore, the combined DSU approach is especially effective for large dynamic systems where many related operations are performed.

Memory usage is also efficient. Both algorithms rely mainly on `parent[]` and `rank[]` arrays, so the space complexity is $O(n)$. Dynamic Connectivity may also store an operation list of size m if operations are not processed as a stream, but the DSU storage itself remains linear. Compared with

storing and repeatedly traversing a full adjacency structure for every query, DSU is compact and practical for incremental connectivity problems.

Algorithm 1 can be used in certain real-world applications, for example, to check the cluster of a node, a system requires a fast component identification. Algorithm 2 is more applicable to systems that evolve with time, for example maintaining graph components, adding road links, monitoring friendships and network connections. In general, Algorithm 1 is an enabling operation, and Algorithm 2 is a more general problem that is solved using Algorithm 1.

5.2 Conclusion

This report evaluated Disjoint Set Union as the chosen data structure and analyzed two related algorithms: Find with Path Compression and Dynamic Connectivity using Union-Find. DSU was shown to be suitable because it represents disjoint components compactly using parent pointers and supports efficient merging and representative lookup.

The correctness demonstrations showed that Find with Path Compression correctly returns the representative of a set and preserves set membership while flattening the parent path. The Dynamic Connectivity trace showed that DSU correctly maintains connectivity relationships as operations are processed, and the invariant-based explanation confirmed that equal representatives correspond to connected components.

The complexity analysis showed that Find with Path Compression has $O(1)$ best-case time and $O(n)$ worst-case time for one isolated operation on a chain. Dynamic Connectivity has $O(1)$ best-case time for a trivial query, $O(\log n)$ conservative worst-case time per operation with union by rank, and $O(m \log n)$ for m operations. The report also acknowledged the stronger amortized $O(m \alpha(n))$ behavior of optimized DSU over long operation sequences.

Data representations and algorithms selected are overall correct, efficient and appropriate for the assignment. They demonstrate how a simple parent-array structure can support both low-level optimization and higher-level problem solving in connectivity-based applications.

Part E: Problem Solving Skill

This work demonstrates problem-solving skill because the selected topic is not limited to a basic array or sorting example. Instead, it connects a specialized data structure with two meaningful algorithmic uses. The report identifies suitable best-case and worst-case data sets, traces the algorithms step by step, and explains how the internal parent structure affects time complexity.

The solution also shows independent reasoning by treating Dynamic Connectivity carefully as an algorithmic procedure solved using DSU. This avoids presenting it as a vague concept and instead defines its input, operations, internal process, and output. The correctness proofs use invariants rather than only descriptive explanation, and the complexity section includes both direct worst-case reasoning and an advanced amortized note. These features strengthen the overall analysis and satisfy the requirement to solve the task with minimal guidelines.

7.0 References

1. cp-algorithms. (2026). Disjoint Set Union. https://cp-algorithms.com/data_structures/disjoint_set_union.html
2. DTU Compute. (2023). Union Find. Technical University of Denmark. <https://courses.compute.dtu.dk/02105/2023/part2/3unionfind/unionfind1x1.pdf>
3. GeeksforGeeks. (2026). Union by Rank and Path Compression in Union-Find Algorithm. <https://www.geeksforgeeks.org/dsa/union-by-rank-and-path-compression-in-union-find-algorithm/>
4. Labuladong. (2026). Union-Find Algorithm. <https://labuladong.online/en/algo/data-structure/union-find/>
5. Linux Kernel Documentation. (2024). Union-Find in Linux. https://docs.kernel.org/core-api/union_find.html

8.0 Appendix A: Requirement Mapping Checklist

Assignment Requirement	Where Addressed in Report
One selected data structure	Disjoint Set Union described in Section 2
Two related algorithms	Find with Path Compression and Dynamic Connectivity represented in Section 3
Best-case and worst-case input data sets	Defined in Section 4
Worst-case step-by-step trace for Algorithm 1	Presented in Section 5
Worst-case step-by-step trace for Algorithm 2	Presented in Section 6
Worst-case complexity with justification	Sections 7.1 and 8.1
Best-case complexity with trace	Sections 7.2-7.3 and 8.2-8.3
Comparative evaluation and conclusion	Sections 9 and 10
Problem-solving skill	Part E

Table 10: Requirement mapping checklist